

Realisatielogboek

[ONDERTITEL VAN DOCUMENT]

GROEP C-1

Inleiding.....	2
Welke ID'es wij gebruiken.....	3
Hoe wij AI gebruiken	4
Pluk Zwaal.....	4
Merge conflicten oplossen met Github CoPilot	4
Gemini gevraagd om openmrs te versimpelen	4
Gemini tussentijdense controles	5
Claude Code die code schreef en pr-dubbel controleerde.....	5
Conclusie.....	6
Junbo Chen	8
Tjeerd Kootwijk	9
Karim Alkichouhi	10
Commits.....	12



INLEIDING

In dit realisatielogboek documenteren wij (Pluk, Junbo, Tjeerd en Karim) het ontwikkelproces van onze C1-applicatie. Het rapport biedt een overzicht van de gemaakte keuzes rondom ontwikkelomgevingen (IDE's) en beschrijft in detail hoe we verschillende AI-tools hebben ingezet als ondersteuning bij het programmeren en het begrijpen van complexe materie zoals OpenMRS en Java. Tot slot geven we inzicht in de werkverdeling aan de hand van onze GitHub-commits.



WELKE IDE'S WIJ GEBRUIKEN

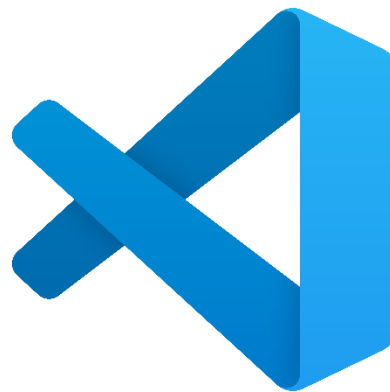
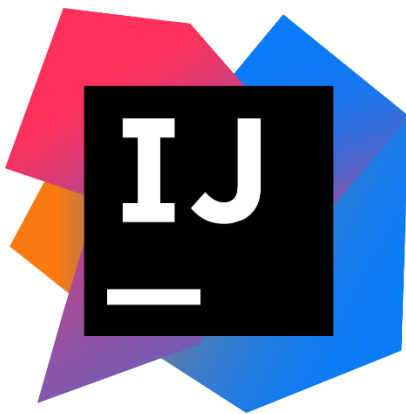
Het team heeft tijdens dit project gebruikgemaakt van twee IDE's: Visual Studio Code (VS Code) en IntelliJ IDEA.

Visual Studio Code

VS Code was voor iedereen de vertrouwde omgeving aan het begin van het project. Het is een licht en snel programma met handige extensies, en omdat we er al ervaring mee hadden, was de drempel laag. Voor de eerste weken van het project werkte dit prima.

IntelliJ IDEA

Naarmate de codebase groeide en de Java-logica complexer werd, stapten we allemaal een voor een over naar IntelliJ IDEA. Sommigen deden dit direct vanaf de start, anderen later in het project. IntelliJ biedt veel betere ondersteuning voor Java: het signaleert fouten direct tijdens het typen, biedt slimme code-aanvulling en maakt het eenvoudig om door te klikken naar methodes en klassen in andere bestanden. Uiteindelijk heeft het hele team de overstap gemaakt, omdat de voordelen voor een Java-project als dit simpelweg te groot waren om te negeren.

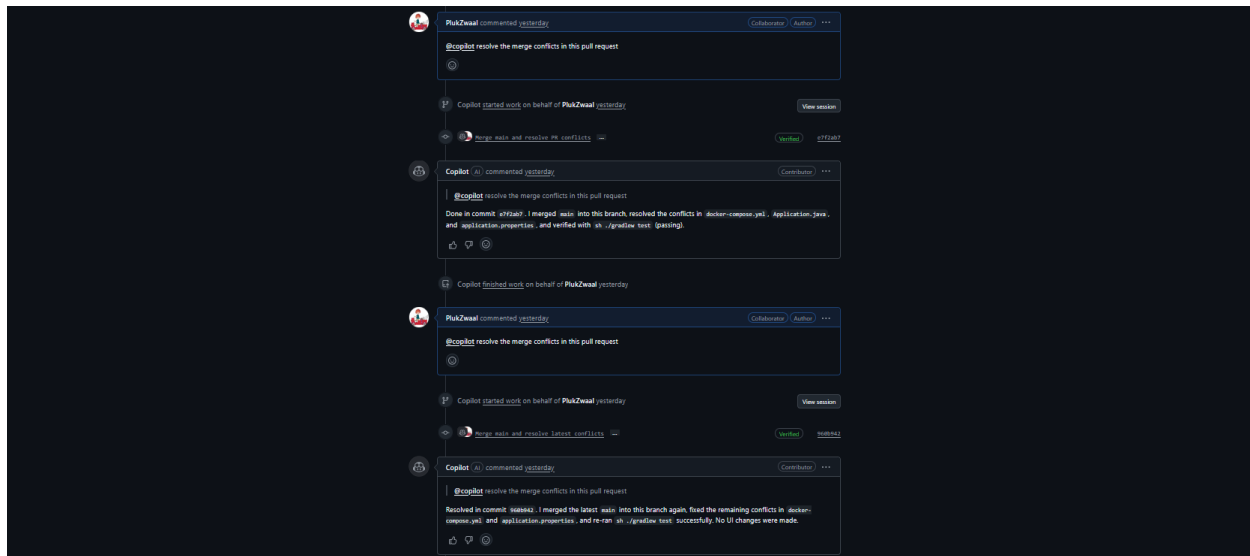


HOE WIJ AI GEBRUIKEN

PLUK ZWAAL

MERGE CONFLICTEN OPLOSSEN MET GITHUB COPILOT

Voordat mijn code naar de main branch gemerged mocht worden, moest ik eerst een aantal merge conflicten oplossen. Dit vind ik zelf altijd een enorm lastig en onoverzichtelijk klusje. Toen ontdekte ik dat ik GitHub Copilot direct in de Pull Request kon gebruiken om me hierbij te helpen. Ik heb dit meteen in de praktijk getest door de AI simpelweg te taggen met de prompt `@copilot resolve the merge conflicts in this pull request`. Wat het direct een stuk betrouwbaarder maakte, is dat je Copilot niet zomaar blindelings de boel laat overschrijven. Je kunt precies het hele denkproces volgen en zien welke stappen hij zet voordat hij de wijzigingen doorvoert.



Toch bleek dat je de controle niet volledig moet loslaten. Bij de eerste poging gaf Copilot namelijk aan dat de conflicten waren opgelost en dat de tests succesvol waren gedraaid, maar bij nadere inspectie had hij eigenlijk maar één van de drie conflicten echt gefixt. Ik heb hem dezelfde opdracht daarom nog een keer gegeven. Bij die tweede poging werkte het wel perfect: hij pakte de resterende conflicten in onder andere docker-compose.yml en de configuratiebestanden op, zonder de bestaande code of functionaliteit kapot te maken. Het was een enorm nuttige ervaring. De AI nam het ingewikkelde werk uit handen, maar het bewees ook dat mijn eigen toezicht als developer absoluut nodig blijft om te garanderen dat de code echt klopt

GEMINI GEVRAAGD OM OPENMRS TE VERSIMPELEN

Toen dit project van start ging, voelde het als een enorm doolhof. Het was naar mijn mening het meest onduidelijke project dat Avans ooit heeft gegeven. Ik was in het begin



echt in shock en wist niet goed waar ik moest beginnen. Vooral het begrijpen van OpenMRS verliep dramatisch. Om de stof van de opdracht toch goed te begrijpen en aan Leeruitkomst 1 (LU1) te voldoen, heb ik Gemini Advanced ingezet. Via een speciale actie voor studenten had ik een jaar lang gratis toegang tot deze premium versie, waardoor ik gebruik kon maken van betere modellen die veel complexer kunnen nadenken. Ik heb de AI in deze fase bewust niet gebruikt om code te schrijven, maar puur als docent voor theorie en architectuur. Ik gebruikte prompts zoals: *"Leg mij eens uit wat OpenMRS is,"* en *"Leg mij uit hoe ik een applicatie kan maken die verbindt met OpenMRS, hoe ziet dit eruit? Schrijf nog geen echte code, geef gewoon uitleg."* Wanneer ik merkte dat de standaard technische documentatie nog steeds te taai of abstract was, paste ik mijn prompts aan om het te versimpelen. Een voorbeeld van een prompt die ik hiervoor gebruikte was: *"Leg de architectuur van OpenMRS uit alsof ik een jongen van 15 ben zonder enige programmeervaring."* Dit vind ik persoonlijk een enorm fijne manier van leren. De AI vermijdt dan alle onnodig moeilijke vaktermen en legt de kern uit in hele duidelijke, begrijpelijke taal. Hierdoor begreep ik de complexe materie ineens wél helemaal.

GEMINI TUSSENTIJDENSE CONTROLES

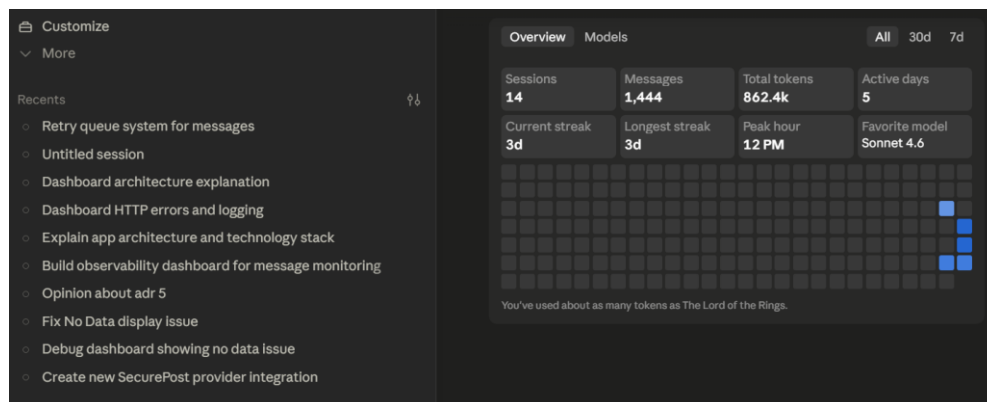
Tijdens het bouwen van de applicatie heb ik Gemini ingezet voor een doorlopende kwaliteits- en architectuurcontrole. Het grote voordeel hiervan was dat ik mijn volledige GitHub repository in één keer naar Gemini kon uploaden, waardoor de AI direct elk bestand en de onderlinge samenhang kon zien. Dit gebruikte ik voor een constante tussentijdse controle van de applicatie terwijl we hem aan het bouwen waren. Ik vroeg Gemini telkens specifiek of onze code nog wel klopte volgens ons C2-model en of we voldeden aan software-eisen zoals het Open/Closed principe. Ook liet ik de AI meekijken of er kwetsbaarheden in de code zaten en stuurde ik de casus mee om te controleren wat we op dat moment nog misten aan functionaliteiten. De belangrijkste reden om dit op deze manier aan te pakken, was de enorme tijdwinst. Als we deze volledige code- en architectuurcontroles handmatig hadden moeten uitvoeren, had dit niet alleen gigantisch veel tijd gekost, maar zouden we in zo'n grote codebase gegarandeerd ook cruciale details of kwetsbaarheden over het hoofd hebben gezien. Door deze controles aan de AI over te laten, bespaarden we tijd, verhoogden we de kwaliteit en wisten we zeker dat we gedurende het programmeren op het juiste spoor bleven.

CLAUDE CODE DIE CODE SCHREEF EN PR-DUBBEL CONTROLEERDE

Wat dit project extra uitdagend maakte, is dat we de applicatie moesten bouwen in een voor ons compleet nieuwe programmeertaal: Java. Hier hadden we echt nog nooit iets mee gedaan. In eerdere periodes maakte ik al regelmatig gebruik van de gewone versie van Claude, maar ik merkte al snel dat dit nu simpelweg niet toereikend was. De onbekende Java-code werd dusdanig complex en verspreid over meerdere bestanden,



dat ik constant tegen de limieten van de gratis versie aanliep. Om die reden heb ik Claude Pro aangeschaft en ben ik intensief met Claude Code gaan werken. Ik heb deze tool direct gekoppeld aan mijn branch, wat werkte als een hele efficiënte vorm van pair programming. Wat hierbij echt een gamechanger was, is dat ik de AI inzette als autonome agents. In plaats van een simpel stukje code te genereren, gaf ik de agent een complexe taak. Hij ging vervolgens zelfstandig aan de slag, schreef de code, testte het, en als hij tegen fouten aanliep, bleef hij uit zichzelf itereren en nieuwe pogingen doen net zolang totdat het daadwerkelijk werkte. Dit proces verliep fantastisch. De werkverdeling was ideaal: de AI deed het harde programmeerwerk en ploeterde door de onbekende syntax heen, terwijl ik de ruimte had om na te denken over de overkoepelende logica. Hierdoor kon ik het overzicht bewaken en garanderen dat de oplossingen netjes pasten bij onze opgestelde ADR's en het C2-model. Tijdens het programmeren kwam Claude ook proactief met tips en wees hij mij op mogelijke problemen in de toekomst waar ik zelf nog niet aan had gedacht. Deze inzichten kon ik rechtstreeks meenemen als waardevolle input voor onze FMEA. Daarnaast was de tool essentieel bij het nakijken van Pull Requests. Ik liet Claude Code de PR's analyseren op mogelijke fouten en risico's. Ik loste deze niet altijd direct op, maar noteerde ze netjes voor de FMEA. De belangrijkste focus bij deze PR-controles was valideren of de code van groepsgenoten daadwerkelijk voldeed aan de afgesproken regels, met name het Open/Closed principe. Hoewel deze manier van werken in de basis erg lijkt op GitHub Copilot, heb ik gemerkt dat Claude Code in de praktijk gewoon vele malen beter en slimmer is in het overzien van het grotere geheel. Als we de syntax van deze nieuwe programmeertaal en alle bijbehorende logica helemaal handmatig hadden moeten uitvogelen, was dat simpelweg onmogelijk geweest qua tijd. Eerlijk gezegd was dit hele project zonder de hulp van AI voor ons gewoon niet te halen.



CONCLUSIE

Als ik terugkijk op dit project, heeft de inzet van AI-tools zoals Gemini, Claude Code en GitHub Copilot echt een onmogelijke opgave veranderd in een haalbaar traject. Zonder deze tools was het simpelweg niet te doen geweest. De toegevoegde waarde was enorm: Gemini gaf me de theoretische basis door de complexe materie van OpenMRS begrijpelijk te maken, Claude Code nam het zware programmeerwerk in Java (wat helemaal nieuw voor ons was) uit handen, en Copilot bracht overzicht bij die lastige merge conflicten. Toch ben ik er ook achter gekomen dat AI niet alleen maar voordelen heeft en er duidelijke "kosten" en risico's aan vastzitten. Ik liep al snel tegen de limieten van de gratis modellen aan, moest extra tijd maken om de code te begrijpen die Claude had geschreven omdat het toch afweek van hoe ik het had gezegd blijkbaar omdat mijn eigen logica achteraf niet helemaal de juiste manier bleek te zijn. En zag met eigen ogen hoe AI kan hallucineren toen Copilot onterecht zei dat alle conflicten waren opgelost. Mijn belangrijkste les is dan ook



dat je eigen toezicht absoluut nodig blijft. Om toekomstige projecten nog efficiënter aan te pakken, neem ik een paar concrete verbeterpunten mee. Ten eerste pas ik mijn prompt-strategie aan. Het trucje om lastige documentatie uit te laten leggen alsof ik een jongen van vijftien ben zonder programmeervaring, werkte zó goed dat ik dit bij onbekende frameworks direct vanaf dag één ga doen, in plaats van pas als ik vastloop. Dat scheelt me gewoon dagen aan verwarring. Ten tweede ga ik bij de start van een project sneller opschalen. Ik verloor in het begin kostbare tijd omdat de gratis AI-limieten er snel doorheen vlogen bij zo'n grote codebase. De volgende keer beslis ik direct in de opstartfase al of we Pro-abonnementen en agents nodig hebben, zodat we halverwege de sprint niet opeens vastlopen. Tot slot hanteer ik in de toekomst een 'zero-trust' beleid bij AI-code. Ik heb gemerkt dat AI soms gewoon fouten maakt of afwijkt van de instructies die ik heb gegeven. Zelfs als een tool met de volle 100% overtuiging zegt dat iets is opgelost, behandel ik het voortaan puur als een junior developer. Dat betekent dat ik de voorgestelde wijzigingen altijd eerst heel goed doorlees, maar de code vooral ook uitgebreid test in de praktijk, voordat ik het echt definitief overneem of een branch merge



JUNBO CHEN

Tijdens dit project heb ik vooral gebruikgemaakt van GitHub Copilot, ChatGPT, grok en claude. Ik heb AI niet gebruikt om complete functionaliteiten te laten bouwen, maar vooral als een zeer snelle en slimme pair-programmer en research-assistent.

Bij het begin van nieuwe onderdelen vroeg ik ChatGPT vaak om een duidelijke uitleg van hoe bepaalde dingen werken binnen OpenMRS, zoals het FHIR/HL7-systeem, de module-structuur of hoe authenticatie en rechten werken. Dit hielp me om snel grip te krijgen op complexe concepten zonder uren in de officiële documentatie te hoeven spitten. Daarna gebruikte ik die kennis om gerichte prompts te maken.

Met GitHub Copilot werkte ik vooral in de IDE. Ik liet het inline suggesties geven bij het schrijven van boilerplate code, service-lagen, controllers en repository-methodes. Wat ik prettig vond is dat ik Copilot vaak gebruikte om refactoring te doen of code netter en consistenter te maken volgens ons C2-model en de ADR's. Ik gaf dan prompts zoals: "Herschrijf deze methode zodat deze het Open/Gesloten-principe volgt en beter onderhoudbaar wordt."

Ik heb bewust altijd zelf de code nagekeken en aangepast. AI was voor mij een accelerant: het zorgde ervoor dat ik sneller kon doorwerken, maar ik bleef verantwoordelijk voor de logica, de integratie met de rest van de applicatie en de kwaliteit. Vooral bij het debuggen van lastige errors was Claude erg waardevol. Ik kon een stacktrace en wat context plakken, waarna de AI vaak al snel de kern van het probleem aanwees.

Conclusie eigen gebruik Door AI op deze manier in te zetten, kon ik veel efficiënter werken en sneller leren werken met Java en OpenMRS — twee dingen die helemaal nieuw voor ons waren. Tegelijkertijd heb ik geleerd dat kritisch blijven kijken naar de gegenereerde code essentieel is. AI maakt soms aannames die niet aansluiten bij ons project of kiest oplossingen die technisch werken maar architecturaal niet optimaal zijn.



TJEERD KOOTWIJK

Tijdens dit project heb ik voornamelijk gebruikgemaakt van ChatGPT en af en toe van GitHub Copilot. Ik gebruikte AI vooral als ondersteuning tijdens het ontwikkelen van het systeem en niet om het volledige project automatisch te laten bouwen. Vaak begon ik met het vragen om een soort startpunt of basisopzet voor een onderdeel. Hierdoor kon ik sneller beginnen en kreeg ik beter inzicht in hoe ik een probleem kon aanpakken. Daarna werkte ik zelf verder aan de code en paste ik de oplossingen aan zodat ze aansloten op de rest van het project.

Naast het genereren van kleine stukken code heb ik AI ook veel gebruikt om concepten beter te begrijpen. Omdat we werkten met nieuwe technologieën en complexe onderdelen binnen OpenMRS. Bijvoorbeeld om uit te leggen waarvoor de hl7/fhir wordt gebruikt en hoe het in elkaar zit. ChatGPT heb ik regelmatig gebruikt om uitleg te krijgen over fouten, architectuurkeuzes en de werking van bepaalde functies. Wanneer ik errors kreeg, gebruikte ik AI vaak als hulpmiddel om sneller te begrijpen waar het probleem zat en welke mogelijke oplossingen er waren. Vervolgens controleerde en testte ik deze oplossingen altijd zelf voordat ik ze toevoegde aan het project.

GitHub Copilot gebruikte ik soms wanneer ik snel kleine aanpassingen wilde doen of repetitieve code moest schrijven. Dit hielp vooral om sneller te werken bij eenvoudige taken. Tijdens het project heb ik geleerd dat AI erg handig is als ondersteuning en leerhulpmiddel, maar dat je als developer zelf kritisch moet blijven kijken naar de gegenereerde code. Niet iedere oplossing werkte direct goed en daarom bleef het belangrijk om de code zelf te begrijpen, aan te passen en uitgebreid te testen. Het is vaak ook dat als je zonder te kijken de code die AI geeft gebruikt het niet werkt en je erg lang bezig kan zijn met problemen oplossen die er niet waren omdat AI-dingen toevoegde die niet hoeven



KARIM ALKICHOUI

Tijdens dit project heb ik gebruikgemaakt van Claude en ChatGPT als primaire AI-tools. Net als de rest van het team was Java voor mij een compleet nieuwe programmeertaal, en ook OpenMRS kende ik vooraf nauwelijks. Ik heb AI ingezet op vier specifieke manieren: voor het opdoen van uitleg en theorie, voor het verkrijgen van concrete codevoorbeelden, voor de opzet van tests en voor het debuggen van fouten.

UITLEG EN THEORIE

Aan het begin van het project heb ik Claude en ChatGPT ingezet om de concepten achter OpenMRS en de HL7/FHIR-standaard te begrijpen. In plaats van uren vast te zitten op de officiële technische documentatie, vroeg ik de AI om uitleg op maat. Voorbeeldprompts die ik hierbij gebruikte:

- Wat is het verschil tussen HL7 v2 en FHIR, en hoe worden ze in OpenMRS toegepast?“
- Uitleg hoe de module-structuur van OpenMRS werkt, in eenvoudige termen.“

Dit hielp me om snel een mentaal model op te bouwen van het systeem voordat ik daadwerkelijk begon te programmeren.

CODEVOORBEELDEN

Wanneer ik een nieuwe functionaliteit moest implementeren, gebruikte ik AI om werkende codevoorbeelden te genereren als startpunt. Ik gaf daarbij altijd de context van ons project mee, zodat de gegenereerde code zo dicht mogelijk bij onze architectuur bleef. Een voorbeeld van een prompt:

- „Geef een voorbeeld van een Java Spring service die een notificatie verstuurt via een externe messaging provider, gebruikmakend van het Strategy-patroon.“

De gegenereerde voorbeelden gebruikte ik nooit blindelings. Ik las de code altijd door, begreep de werking en paste het aan zodat het klopte met onze ADR's en het C2-model. AI leverde daarmee een startpunt, maar het eindresultaat was altijd mijn eigen werk.

TESTOPZET



Het schrijven van tests was voor mij nieuw in combinatie met Java en JUnit. Ik heb AI gebruikt om de opzet van testklassen te begrijpen en als hulp bij het structureren van unit- en integratietests. Voorbeeldprompt:

- „Hoe schrijf ik een JUnit-test voor een service die een notificatie verstuurt, inclusief een mock van de externe provider?“

Hiermee kreeg ik snel inzicht in de teststructuur die past bij onze codebase. Vervolgens schreef ik de daadwerkelijke testgevallen zelf op basis van de functionele eisen uit de casus.

Debuggen en errors

Tijdens het programmeren liep ik regelmatig tegen fouten aan die ik zelf moeilijk kon plaatsen. In die gevallen plakte ik de stacktrace en relevante context in de chat, waarna de AI een analyse gaf van wat er waarschijnlijk mis was. Dit gaf me een breder perspectief op het probleem en hielp me sneller de juiste richting in te slaan. De uiteindelijke fix voerde ik altijd zelf uit en testte ik lokaal voordat ik iets committete.

Mijn algemene conclusie is dat AI als tool enorm waardevol is geweest, maar dat het kritische denkvermogen van de developer onmisbaar blijft. AI geeft een richting, maar jij bent degene die beoordeelt of die richting klopt voor jouw specifieke situatie.

Aantal voorbeeld prompts die ik heb gebruikt:

FHIR/HL7:

- "Wat is HL7 FHIR en waarom gebruiken ziekenhuizen dit? Leg het uit zonder vakjargon."
- "Wat is een FHIR Resource? Geef een voorbeeld van hoe een afspraak (appointment) eruitziet als FHIR JSON."
- "Wat is het verschil tussen HL7 v2 en FHIR R4, en welke gebruiken moderne systemen?"
- "Heeft FHIR R5 wel afspraken appointment?"

Communicatiemodule / architectuur begrijpen:

- "Ik moet een communicatiemodule bouwen die notificaties stuurt via meerdere messaging providers. Welke architectuurpatronen zijn hier geschikt voor?"
- "Wat is het Strategy-patroon en hoe zou ik dat toepassen als ik SwiftSend, LegacyLink en AsyncFlow als losse providers wil implementeren?"
- "Hoe werkt het Open/Closed principe in de praktijk? Geef een voorbeeld in Java waarbij ik een nieuwe messaging provider kan toevoegen zonder bestaande code aan te passen."



COMMITTS

Hieronder vind je een overzicht van de code-bijdragen van het team. Je ziet hierin hoe vaak we iets hebben gepusht (commits) en hoeveel regels code er zijn toegevoegd en verwijderd. Als je naar de cijfers kijkt, valt meteen op dat Karim een stuk meer commits heeft gedaan en meer code heeft geschreven dan de rest. Toch waren de taken onderling wel ongeveer netjes verdeeld. Iedereen heeft echt deel uitgemaakt van de bouw van de hele applicatie en z'n steentje bijgedragen. Het verschil in de statistieken zit hem er vooral in dat de één gewoon wat meer code heeft zitten typen, terwijl de ander zich meer bezighield met taken buiten de code om. Dat soort werk zie je niet direct terug in Git, maar is natuurlijk wel superbelangrijk voor het project!



Link naar GitHub: <https://github.com/TjeerdKootwijk/ATx2.4SoftwarearchitectuurKwaliteit>



